Git Workflow Guide

Betsybots · DIY Challenge Repo · Developer Reference

Overview

This document defines how we manage code in the **DIY-Challenge-Repo** monorepo. All developers follow the same workflow to keep **main** stable and always launchable.

Key Principles

- main is always launchable — never push broken code directly
- Every change goes through a Pull Request (PR) with at least 1 approval
- Small, focused PRs — one task per PR, easy to review
- Task-based branches — own the task end-to-end
- No folder-level ownership — touch whatever files the task needs

Repository Setup

Initial Clone

Clone with submodules:

```
$ git clone --recurse-submodules \
https://github.com/Betsybots/DIY-Challenge-Repo.git
$ cd DIY-Challenge-Repo
```

If you already cloned without submodules:

```
$ git submodule update --init --recursive
```

Verify Your Setup

```
$ git status # should be on main, clean tree
$ git submodule status # all submodules show a commit SHA
$ git remote -v # origin → github.com/Betsybots/...
```

Branching Strategy

We use **short-lived feature branches** off main. No long-running develop branch.

Branch Naming Convention

Type	Pattern	Example
New feature	feature/description	feature/lidar-cluster-classifier

Calibration work	calib/description	calib/imu-noise-params
Bug fix	fix/description	fix/ekf-divergence-on-startup
Config tuning	tune/description	tune/nav2-inflation-radius
Documentation	docs/description	docs/race-day-runbook

Creating a Branch

```
# Always start from latest main
$ git checkout main
$ git pull origin main

# Create your branch
$ git checkout -b feature/my-task-name
```

Daily Workflow

1. Start of Session

```
# Make sure you have latest main merged into your branch
$ git checkout feature/my-task
$ git fetch origin
$ git rebase origin/main

# If rebase has conflicts:
$ git status # see conflicted files
# ... fix conflicts in editor ...
$ git add <fixed-files>
$ git rebase --continue
```

2. Making Changes

Commit often with clear messages. Each commit should be a logical unit of work.

```
$ git add -p # stage hunks interactively (preferred)
$ git add src/my_package/

$ git commit -m "feat(localization): add RTK fusion to EKF config"
```

Commit Message Format

Use conventional commit prefixes:

Prefix	Use For
feat:	New functionality or capability
fix:	Bug fix or correction
calib:	Calibration data or config update

tune:	Parameter tuning (Nav2, EKF, etc.)
docs:	Documentation changes
refactor:	Code restructuring (no behavior change)
chore:	Build, CI, or repo maintenance

Good commit message examples:

```
feat(perception): add PCL cluster size filter for obstacle detection
calib(imu): update noise density from calibration session 2026-06-03
fix(cmd_vel_mux): prevent zero-velocity output when e-stop releases
tune(nav2): reduce inflation radius from 0.55 to 0.40 for narrow pass
```

3. Pushing Your Branch

```
$ git push origin feature/my-task

# First push — set upstream:
$ git push -u origin feature/my-task
```

Pull Requests (PRs)

Every change reaches main via a PR on GitHub. No direct pushes to main are allowed.

Opening a PR

- Go to GitHub → Pull Requests → New Pull Request
- Base: **main** ← Compare: **your-branch**
- Fill in the PR template (see below)
- Assign at least 1 reviewer
- Link any related issues if applicable

PR Description Template

Copy this into your PR description:

```
## What Changed
- Brief list of changes

## Why
- Motivation / context

## How I Tested
- What you ran to validate (launch command, bag replay, etc.)
- Include relevant terminal output or screenshots if helpful

## Known Risks
- Any areas of concern or things to watch after merge

## Checklist
- [ ] Builds clean: colcon build --packages-select <pkg>
- [ ] Tested on robot / in sim / bag replay (pick one)
- [ ] No TF errors or topic mismatches in logs
- [ ] Config changes documented in commit message
```

Review Process

- Reviewer has **24 hours** to review (48h max if weekend)
- Reviewer checks: does it build? does the description make sense? any obvious issues?
- Reviewer can **Approve**, **Request Changes**, or **Comment**
- If changes requested → push fixes to the same branch → re-request review
- Once approved → the PR author merges

Merging

Use **Squash and Merge** for most PRs (keeps main history clean). Use **Merge Commit** only for large multi-commit PRs where individual commits matter.

```
# After merge on GitHub, clean up locally:
$ git checkout main
```

```
$ git pull origin main
$ git branch -d feature/my-task # delete local branch
```

■ **Never force-push to main or rewrite main history.**

Branch Protection Rules

These rules are configured on GitHub for the **main** branch:

Rule	Setting
Require PR before merge	Yes — no direct pushes
Required approvals	1 minimum
Dismiss stale reviews on new push	Yes
Require branches up-to-date	Yes — must rebase before merge
Force push	Blocked for everyone
Branch deletion	Blocked (main only)

Handling Merge Conflicts

Conflicts happen when two branches edit the same lines. Here's how to resolve:

Option A: Rebase (preferred)

```
$ git fetch origin
$ git rebase origin/main

# For each conflict:
$ git status # see which files conflict
# Edit the files — remove <<</===/>>> markers
$ git add <resolved-file>
$ git rebase --continue

# If it gets messy, abort and start over:
$ git rebase --abort
```

Option B: Merge (if rebase is complex)

```
$ git fetch origin
$ git merge origin/main
# Resolve conflicts same way, then:
$ git commit # creates merge commit
```

When to Ask for Help

- If a conflict touches files you didn't write — ping that developer

- If rebase produces more than 3 conflicted files — discuss in sync meeting
- Never force-push over someone else's commits without telling them

Submodule Management & Patches

The **third_party_ws/src/** folder contains 6 git submodules — third-party ROS 2 packages that we use as-is or with small modifications applied via patch files.

Submodule	Purpose	Patched?
FAST_LIO	Real-time LiDAR-inertial odometry	No
LIO-SAM	Offline prior map generation	No
imu_utils_ros2_humble	IMU noise calibration tool	No
lidar_imu_calib	LiDAR-IMU extrinsic calibration	Yes
livox_ros_driver2	Hesai QT64 LiDAR driver	Yes
ndt_omp_ros2	NDT scan matching (localization)	Yes

How Submodules Work in This Repo

- Each submodule points to a **specific upstream commit SHA** — a known-good version
- The SHA is tracked in the parent repo (like a bookmark to that exact version)
- When you clone with **--recurse-submodules**, git checks out that exact commit
- We **never modify code directly inside submodule directories**
- If we need changes, we use **patch files** applied at build time

Why Not Just Edit the Submodule Directly?

- Commits inside a submodule only exist on YOUR machine until pushed to the upstream remote
- If you commit inside a submodule, the parent repo records a SHA that nobody else has
- Result: teammates clone the repo and git fails to find the submodule commit → broken checkout
- Patches avoid this entirely — the modification lives in OUR repo as a .patch file

The Patch System

Patches are small diff files stored in the **patches/** directory. They record changes we need on top of the upstream submodule code.

Current Patches

Patch File	Applied To	What It Does
lidar_imu_calib.patch	lidar_imu_calib	Build fixes for ROS 2 Humble compatibility
livox_ros_driver2.patch	livox_ros_driver2	Config adjustments for Hesai QT64

ndt_omp_ros2.patch	ndt_omp_ros2	Build fixes and parameter changes for our setup

Applying Patches (After Clone)

Patches are applied by **setup.sh** or manually:

```
# Apply a single patch:
$ cd third_party_ws/src/livox_ros_driver2
$ git apply ../../../patches/livox_ros_driver2.patch

# Or apply all patches at once (setup.sh does this):
$ cd /path/to/DIY-Challenge-Repo
$ bash setup.sh
```

Note: After patches are applied, the submodule directory is "dirty" (has uncommitted changes). This is expected and .gitmodules has ignore = dirty to suppress the noise.

Creating a New Patch

When you need to modify a submodule (e.g., fix a build issue, change a default param):

```
# 1. Make your edits inside the submodule
$ cd third_party_ws/src/FAST_LIO
$ nano src/laserMapping.cpp # or whatever file needs changing

# 2. Generate the patch file
$ git diff > ../../../patches/fast_lio.patch

# 3. Go back to the parent repo and commit the PATCH (not the submodule)
$ cd ../../..
$ git add patches/fast_lio.patch
$ git commit -m "fix(submodule): patch FAST_LIO for <reason>"
```

■ Important: Do NOT run git add third_party_ws/src/FAST_LIO — that would change the submodule pointer to a local-only commit.

Updating a Patch

If a patch needs changes (e.g., you need an additional fix in the same submodule):

```
# 1. Reset the submodule to upstream state
$ cd third_party_ws/src/livox_ros_driver2
$ git checkout .

# 2. Apply the existing patch
$ git apply ../../../patches/livox_ros_driver2.patch

# 3. Make your additional edits
$ nano src/some_file.cpp

# 4. Regenerate the patch (captures ALL changes vs upstream)
$ git diff > ../../../patches/livox_ros_driver2.patch
```

```
# 5. Commit the updated patch
$ cd ../../..
$ git add patches/livox_ros_driver2.patch
$ git commit -m "fix(submodule): update livox_ros_driver2 patch for <reason>"
```

Updating a Submodule to a Newer Upstream Version

If upstream publishes a bug fix or new feature we need:

```
# 1. Enter the submodule and fetch upstream
$ cd third_party_ws/src/FAST_LIO
$ git fetch origin
$ git log --oneline origin/main -5 # see recent commits

# 2. Move to the new commit
$ git checkout <new-commit-sha>

# 3. Check if existing patch still applies (if one exists)
$ git apply --check ../../../patches/fast_lio.patch
# If it fails: regenerate or fix the patch manually

# 4. Go to parent repo and commit the pointer update
$ cd ../../..
$ git add third_party_ws/src/FAST_LIO
$ git commit -m "chore(submodule): update FAST_LIO to <sha> (reason)"
```

.gitmodules and ignore = dirty

Our **.gitmodules** file includes **ignore = dirty** for patched submodules:

```
[submodule "third_party_ws/src/livox_ros_driver2"]
path = third_party_ws/src/livox_ros_driver2
url = https://github.com/...
ignore = dirty
```

- This tells **git status** to ignore uncommitted changes inside the submodule
- Without this, every dev would see "modified: third_party_ws/src/..." after applying patches
- It does **NOT** ignore submodule pointer changes — those still show up correctly

Submodule Golden Rules

- Never git add a submodule directory unless you intend to update its pointer
- Never git commit inside a submodule — your teammates can't access those commits
- Always use patch files for modifications — commit the .patch, not the submodule
- After cloning, run setup.sh to apply all patches automatically
- If a patch fails to apply after a submodule update, fix the patch and commit the fix

Common Scenarios

Scenario: Wrong branch — committed to main

```
# Move the commit to a new branch (if not pushed yet)
$ git branch feature/oops # create branch at current HEAD
$ git reset --hard HEAD~1 # move main back one commit
$ git checkout feature/oops # continue on new branch
```

Scenario: Need to undo last commit (not pushed)

```
$ git reset --soft HEAD~1 # undo commit, keep changes staged
# or
$ git reset --mixed HEAD~1 # undo commit, unstage changes
```

Scenario: Stash work to switch branches

```
$ git stash # save current changes
$ git checkout other-branch
# ... do something ...
$ git checkout feature/my-task
$ git stash pop # restore changes
```

Scenario: PR is behind main

GitHub will show "This branch is behind main" — you need to rebase:

```
$ git fetch origin
$ git rebase origin/main
$ git push --force-with-lease # safe force push (only your branch)
```

Note: --force-with-lease is safe for your own feature branch. Never use --force on main or shared branches.

Quick Reference Card

Action	Command
Start new task	git checkout main && git pull && git checkout -b feature
Stage changes	git add -p (interactive) or git add <files>
Commit	git commit -m "type(scope): description"
Push branch	git push -u origin feature/name
Update from main	git fetch origin && git rebase origin/main
Resolve conflicts	Edit files → git add → git rebase --continue
Force push (own branch)	git push --force-with-lease
After PR merged	git checkout main && git pull && git branch -d feature/n
Stash work	git stash / git stash pop
View log	git log --oneline -20
See what changed	git diff --stat origin/main

Golden Rules

- Never push directly to main
- Never force-push to main or someone else's branch
- Always rebase before merge — keep history linear
- One task per branch, one outcome per PR
- If in doubt, ask in the weekly sync

— Betsybots Team · DIY Challenge 2026